34. Host Hardening Checklist

34. Host Hardening Checklist

Phase: Phase 9 — Security and SSH
Estimated time: 1–2 days
Slug: `34-hardening-basics`

Learning objectives

- Apply a practical hardening baseline
- Disable unused services
- Enforce SSH key auth
- Keep patching cadence

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Most compromises of ordinary servers are not clever. Attackers scan the internet for password SSH, outdated packages, open databases, and forgotten admin accounts. A small, repeatable **hardening baseline** removes the cheap attack paths and costs an hour. This module defines a five item baseline you can apply to any new Linux host, then automate (see the Ansible module).

Core concepts

The baseline in five items:

#	Item	Goal
1	ssh-keys	Key-only login, no root login, no passwords
2	firewall	Default deny inbound, allow only needed ports
3	updates	Security patches applied automatically or on schedule
4	sudo	Named admin users, least privilege, no shared root
5	ntp	Accurate time for logs, TLS, and Kerberos

```
internet --> [firewall: allow 22,443 only] --> [sshd: keys only]
                                                      |
                                            [sudo: named admins, logged]
                                                      |
                                  [patched packages] + [synced clock + logs]
```

Principles behind the list:

- **Reduce attack surface**: fewer open ports, fewer installed packages, fewer running services.
- **Least privilege**: users and services get the minimum rights.
- **Defense in depth**: several independent layers, none assumed perfect.
- **Detect and recover**: logs with correct time, backups, and alerts.

Hardening is not a one-time event; it is a baseline plus periodic checking.

Worked examples

1. SSH keys only. Edit `/etc/ssh/sshd_config.d/10-hardening.conf`:

```
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
MaxAuthTries 3
AllowUsers deploy alice
```

Validate before reloading, and keep a second session open:

```
sudo sshd -t && sudo systemctl reload ssh
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no host   #
should fail
```

2. Firewall (Ubuntu with ufw):

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose
```

Or with firewalld:

```
sudo firewall-cmd --permanent --add-service=ssh
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --reload
```

3. Automatic security updates:

```
sudo apt install unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
systemctl status unattended-upgrades --no-pager
# RHEL family: dnf-automatic
```

4. sudo hygiene:

```
sudo visudo -f /etc/sudoers.d/admins     # always visudo, never plain edit
# %admins ALL=(ALL) ALL
getent group sudo
sudo passwd -l root                       # lock root password login
```

5. Time sync:

```
timedatectl
systemctl status systemd-timesyncd chrony --no-pager 2>/dev/null
chronyc tracking
```

Audit what is listening and what is installed:

```
ss -ltnup
systemctl list-unit-files --state=enabled | head -n 30
dpkg -l | wc -l
```

A tiny checker that counts how many baseline items pass:

```
n=0
grep -Rqs '^PasswordAuthentication no' /etc/ssh/sshd_config /etc/ssh/
sshd_config.d && n=$((n+1))
ufw status 2>/dev/null | grep -q 'Status: active' && n=$((n+1))
systemctl is-enabled unattended-upgrades >/dev/null 2>&1 && n=$((n+1))
getent group sudo >/dev/null && n=$((n+1))
timedatectl show -p NTPSynchronized --value | grep -q yes && n=$((n+1))
echo "baseline items passing: $n/5"
```

Mini-lab

1. On a throwaway VM, record `ss -ltnup` before changes.
2. Apply the five items one at a time, testing after each.
3. Confirm password SSH fails, an extra port (for example 8080) is blocked from another host, and `timedatectl` shows synchronized.
4. Run `lynis audit system` (if installable) and read the top suggestions.
5. Write the checker above as `baseline.sh` and run it before and after.

Decision guide / failure modes

Question	Answer
Port must be public?	If not, bind to localhost or private network
Many admins?	Individual accounts + sudo groups, never shared logins
Reboot for kernel patches?	Schedule it; consider live patching where available
Cloud host?	Also use provider security groups; defense in depth
Brute force noise?	Keys only; fail2ban is optional, not primary

Failure modes:

- **Locking yourself out**: enabling a firewall or disabling passwords without testing a second session. Always test from a new terminal.
- **Firewall allow after enable**: allow SSH before turning it on.
- **Config drift**: manual changes diverge over time; enforce with automation.
- **Updates without restart**: services keep running old libraries; check `needrestart` or `checkrestart`.
- **Wrong time**: drifting clocks break TLS and make log correlation useless.
- **Hardening theater**: ticking boxes while leaving a database open on 0.0.0.0.

Common mistakes

- Blindly running anonymous harden scripts
- Locking yourself out of SSH
- Assuming compliance checkbox equals secure

Exercises

Work in `exercises/34-hardening-basics/`. Read that folder's `README.md` fully before coding.

```
cd exercises/34-hardening-basics
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement `count_baseline_items` in `exercise.sh` so it prints the number of items in the hardening baseline from this lesson: ssh-keys, firewall, updates, sudo, and ntp. Print just the number, no words. Do the counting in your head from the table, not from the host. Afterwards, write a one-line check for each item on your own VM and record which ones fail. Keep the function body to one line; the goal is recall of the lesson vocabulary, not cleverness. Finish by re-reading the Decision guide table and noting one failure mode you could have caused yourself.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/34-hardening-basics/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Name five baseline items
2. Why disable unused services?

3. Risk of remote harden scripts?

Further reading

- https://www.cisecurity.org/